PROJECT PRESENTATION

AI IMAGE

1 احمد حاتم ابراهيم احمد

2 محمود علي احمد الصاوي

3 صلاح الدين امير احمد الشبكشي

Description :-

🎨 مُولِّد الصور بالذكاء الاصطناعي
حوّل خيالك لصور واقعية في ثوانٍ! تطبيق بسيط وقوي بيخليك تولّد صور احترافية بالذكاء الاصطناعي من مجرد وصف نصي.

✨ المميزات :-
🖌️ 5 نماذج ذكاء اصطناعي مختلفة كل نموذج بيدّيك أسلوب وجودة مختلفة.
🎭 أنماط فنية جاهزة من لمسة واحدة وتحدد الستايل اللي عايزه.
📚 سجل كامل لصورك.
⬇️ تحميل مباشر احفظ أي صورة على جهازك بضغطة واحدة.
🔐 حسابات آمنة تسجيل دخول وإنشاء حساب باستخدام Firebase Authentication.
⏱️ إدارة ذكية للطلبات مؤقت انتظار بين كل توليد يضمن استقرار الأداء وجودة النتائج.

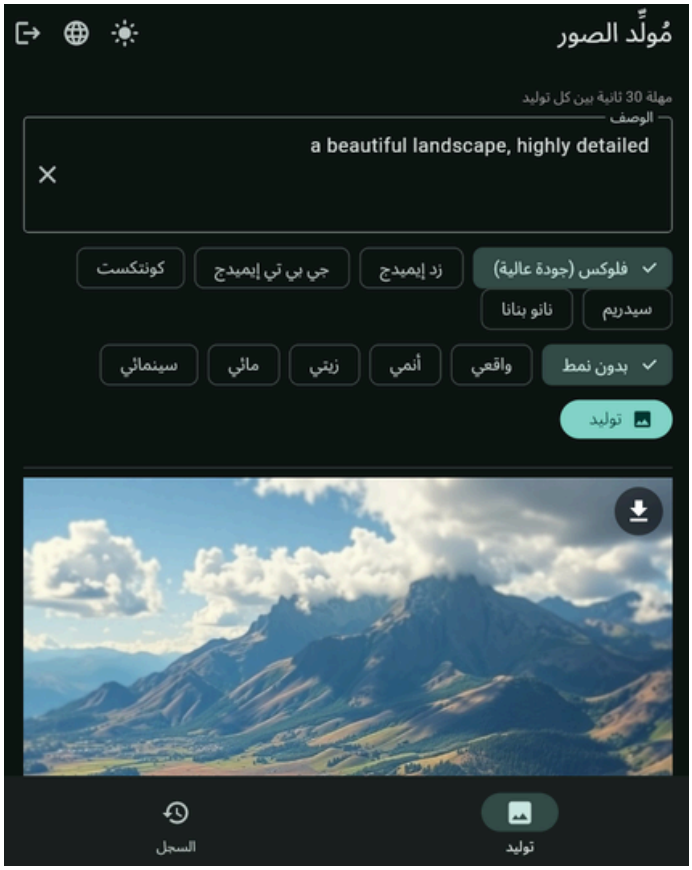
🎨 AI Image Generator
Turn your imagination into stunning visuals in seconds! A simple yet powerful app that lets you generate professional AI images from just a text description.

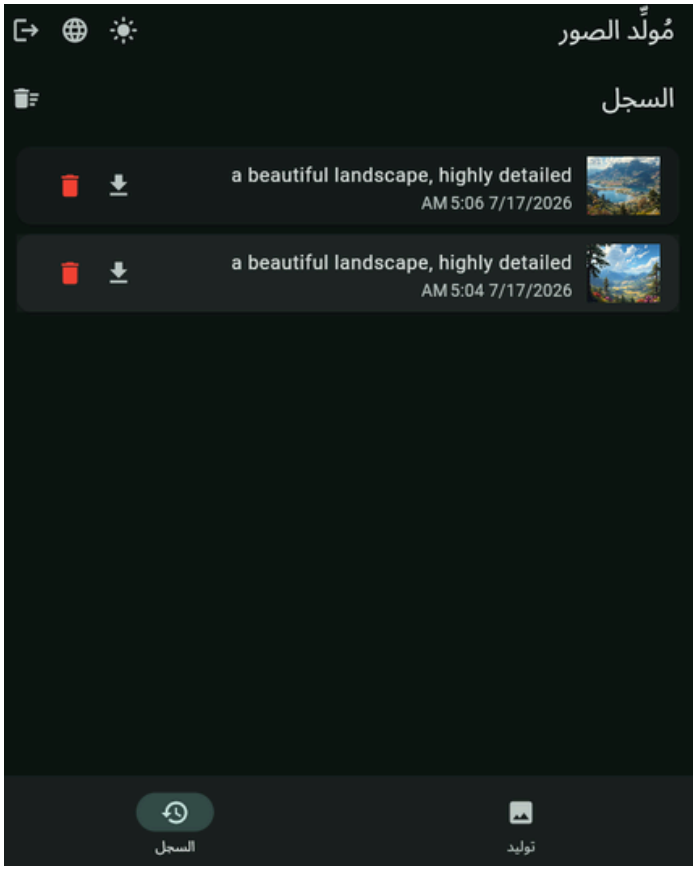
✨ Features :-
🖌️ 5 Different AI Models, Each model gives you a unique style and quality.
🎭 Ready-Made Art Styles, Pick your style with a single tap.
📚 Full Image History.
⬇️ Direct Download, Save any image to your device with one tap.
🔐 Secure Accounts, Sign in and sign up powered by Firebase Authentication.
⏱️ Smart Request Management, A cooldown timer between generations ensures stable performance and quality results.

Click

Download -> Mediafire Github

Screenshots :-





Code :-

```dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:typed_data';
import 'dart:ui' as ui;
import 'package:file_picker/file_picker.dart';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import 'package:intl/date_symbol_data_local.dart';
import 'package:path_provider/path_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

// ضع المفتاح عبر --dart-define=POLLINATIONS_API_KEY=xxxx (لا تكتبه هنا مباشرة)
class Cfg {
  static const url = 'https://image.pollinations.ai/prompt/';
  static const key = String.fromEnvironment('POLLINATIONS_API_KEY');
  static const w = 1024, h = 1024, cooldown = 30, timeout = 120;
}

const dict = {
  'app_title': ['مُولِّد الصور', 'Image Generator'],
  'prompt': ['الوصف', 'Prompt'],
  'generate': ['توليد', 'Generate'],
  'history': ['السجل', 'History'],
  'no_images': ['لا توجد صور بعد', 'No images yet'],
  'delete': ['حذف', 'Delete'],
  'clear_history': ['مسح السجل', 'Clear History'],
  'cancel': ['إلغاء', 'Cancel'],
  'are_you_sure': ['هل أنت متأكد؟', 'Are you sure?'],
  'close': ['إغلاق', 'Close'],
  'arabic_warning': ['اكتب الوصف بالإنجليزية فقط', 'Write the prompt in English only'],
  'cooldown_wait': ['يرجى الانتظار', 'Please wait'],
  'seconds_suffix': ['ثانية', 's'],
  'cooldown_info': ['مهلة 30 ثانية بين كل توليد', 'Model cooldown: 30 seconds'],
  'error': ['خطأ', 'Error'],
  'logout': ['تسجيل الخروج', 'Logout'],
  'login': ['تسجيل الدخول', 'Login'],
  'signup': ['إنشاء حساب', 'Sign up'],
  'email': ['البريد الإلكتروني', 'Email'],
  'password': ['كلمة المرور', 'Password'],
  'have_account': ['لديك حساب؟ دخول', 'Have an account? Login'],
  'no_account': ['لا يوجد حساب؟ إنشاء', 'No account? Sign up'],
  'download': ['تنزيل', 'Download'],
  'saved_success': ['تم الحفظ', 'Saved'],
  'save_cancelled': ['تم الإلغاء', 'Cancelled'],
  'save_failed': ['فشل الحفظ', 'Save failed'],
};

const models = {
  'flux': ['فلوكس', 'Flux'],
  'turbo': ['توربو', 'Turbo'],
  'flux-realism': ['واقعي', 'Realism'],
  'flux-anime': ['أنمي', 'Anime'],
  'flux-3d': ['ثري دي', '3D'],
};

const styles = {
  'none': ['بدون', 'None'],
  'realistic': ['واقعي', 'Realistic'],
  'anime': ['أنمي', 'Anime'],
  'oil-painting': ['زيتي', 'Oil'],
  'watercolor': ['مائي', 'Watercolor'],
  'cinematic': ['سينمائي', 'Cinematic'],
};
```

```
const styleEn = {
  'none': '',
  'realistic': 'realistic',
  'anime': 'anime',
  'oil-painting': 'oil painting',
  'watercolor': 'watercolor',
  'cinematic': 'cinematic',
};

String tr(BuildContext c, String k) =>
    (dict[k] ?? models[k.replaceFirst('model_', '')] ?? styles[k.replaceFirst('style_', '')])!
        [c.watch<AppState>().isAr ? 0 : 1];
bool hasArabic(String t) => RegExp(r'[\u0600-\u06FF]').hasMatch(t);
String cut(String t, int n) => t.length <= n ? t : '${t.substring(0, n)}...';

class GenImg {
  final String path, prompt; final DateTime time; Uint8List? _cache;
  GenImg(this.path, this.prompt, this.time);
  Future<Uint8List> get bytes async => _cache ??= await File(path).readAsBytes();
  Map<String, dynamic> toJson() => {'path': path, 'prompt': prompt, 'time': time.toIso8601String()};
  factory GenImg.fromJson(Map<String, dynamic> j) => GenImg(j['path'], j['prompt'], DateTime.parse(j['time']));
}

Future<String> saveImageFile(Uint8List bytes) async {
  final dir = Directory('${(await getApplicationDocumentsDirectory()).path}/gen_images');
  if (!await dir.exists()) await dir.create(recursive: true);
  final path = '${dir.path}/img_${DateTime.now().millisecondsSinceEpoch}.png';
  await File(path).writeAsBytes(bytes);
  return path; }

class AppState extends ChangeNotifier {
  bool isAr = true, isDark = false, loading = false, ready = false;
  String model = 'flux', style = 'none';
  int cooldownLeft = 0;
  final history = <GenImg>[];
  Timer? _t;
  AppState() { _load(); }

  Future<void> _load() async {
    final p = await SharedPreferences.getInstance();
    isAr = p.getBool('isAr') ?? true; isDark = p.getBool('isDark') ?? false;
    model = models.containsKey(p.getString('model')) ? p.getString('model')! : 'flux';
    style = styles.containsKey(p.getString('style')) ? p.getString('style')! : 'none';
    final raw = p.getString('history');
    if (raw != null && raw.isNotEmpty) {
      try { for (final e in (jsonDecode(raw) as List)) { final img = GenImg.fromJson(e); if (await
File(img.path).exists()) history.add(img); } } catch (_) {} }
    ready = true; notifyListeners(); }

  Future<void> _savePrefs() async {
    final p = await SharedPreferences.getInstance();
    await p.setBool('isAr', isAr); await p.setBool('isDark', isDark); await p.setString('model', model); await
p.setString('style', style); }

  Future<void> _saveHistory() async {
    final p = await SharedPreferences.getInstance();
    await p.setString('history', jsonEncode(history.map((e) => e.toJson()).toList())); }

  void set(void Function() f) { f(); notifyListeners(); _savePrefs(); }
  Future<void> addToHistory(GenImg img) async { history.insert(0, img); notifyListeners(); await
_saveHistory(); }

  Future<void> removeFromHistory(int i) async {
    final img = history.removeAt(i); notifyListeners(); await _saveHistory();
    try { await File(img.path).delete(); } catch (_) {} }

  Future<void> clearHistory() async {
    final old = List<GenImg>.from(history); history.clear(); notifyListeners(); await _saveHistory();
    for (final img in old) { try { await File(img.path).delete(); } catch (_) {} } }

  void startCooldown() {
    cooldownLeft = Cfg.cooldown; _t?.cancel();
    _t = Timer.periodic(const Duration(seconds: 1), (t) { cooldownLeft--; if (cooldownLeft <= 0) { cooldownLeft
= 0; t.cancel(); } notifyListeners(); }); }
```

```
@override void dispose() { _t?.cancel(); super.dispose(); } }

class ApiException implements Exception { final String message; ApiException(this.message); @override String
toString() => message; }

class Api {
  Future<Uint8List> gen(String prompt, String model, String style) async {
    final se = styleEn[style] ?? '';
    final p = '$prompt${se.isNotEmpty ? ', $se style' : ''}, highly detailed, sharp focus, high quality';

    Object? lastErr;
    for (final m in [model, ...models.keys.where((k) => k != model)]) { try { return await _request(p, m); }
catch (e) { lastErr = e; } }
    throw ApiException('فشل التوليد بكل الموديلات المتاحة.\n($lastErr)'); }

  Future<Uint8List> _request(String prompt, String model) async {
    final url = Uri.parse('${Cfg.url}${Uri.encodeComponent(prompt)}').replace(queryParameters: {
      'model': model, 'width': '${Cfg.w}', 'height': '${Cfg.h}', 'nologo': 'true', 'enhance': 'true',
      'seed': '${DateTime.now().millisecondsSinceEpoch % 1000000}', 'referrer': 'image_generate',
    });
    final headers = {'User-Agent': 'Mozilla/5.0 (compatible; FlutterImageApp/1.0)', 'Accept': 'image/*', if
(Cfg.key.isNotEmpty) 'Authorization': 'Bearer ${Cfg.key}'};
    http.Response r;
    try { r = await http.get(url, headers: headers).timeout(Duration(seconds: Cfg.timeout)); }
    catch (e) { throw ApiException('تعذّر الاتصال بالخادم.\n($e)'); }
    if (r.statusCode == 200 && r.bodyBytes.length > 1024 && (r.headers['content-type'] ??
'').contains('image')) return r.bodyBytes;
    throw ApiException('فشل التوليد (HTTP ${r.statusCode})\n${cut(utf8.decode(r.bodyBytes, allowMalformed: true),
200)}'); } }

class AuthService {
  final _auth = FirebaseAuth.instance;
  Stream<User?> get authStateChanges => _auth.authStateChanges();
  Future<String?> signUp(String e, String p) async { try { await _auth.createUserWithEmailAndPassword(email: e,
password: p); return null; } on FirebaseAuthException catch (ex) { return ex.message ?? ex.code; } }

  Future<String?> signIn(String e, String p) async { try { await _auth.signInWithEmailAndPassword(email: e,
password: p); return null; } on FirebaseAuthException catch (ex) { return ex.message ?? ex.code; } }

  Future<void> signOut() => _auth.signOut(); }

final authService = AuthService();

Future<void> downloadImage(BuildContext c, Uint8List bytes, String prompt) async {
  final name = prompt.trim().isEmpty ? 'image' : prompt.replaceAll(RegExp(r'[^a-zA-Z0-9]+'), '_');
  try {
    final path = await FilePicker.platform.saveFile(dialogTitle: tr(c, 'download'), fileName: '${cut(name,
40)}_${DateTime.now().millisecondsSinceEpoch}.png', bytes: bytes, type: FileType.custom, allowedExtensions:
['png']);
    if (c.mounted) ScaffoldMessenger.of(c).showSnackBar(SnackBar(content: Text(tr(c, path == null ?
'save_cancelled' : 'saved_success'))));
  } catch (e) { if (c.mounted) ScaffoldMessenger.of(c).showSnackBar(SnackBar(content: Text('${tr(c,
'save_failed')}: ${cut(e.toString(), 100)}'))); } }

Future<void> confirmDialog(BuildContext c, String title, VoidCallback onOk) => showDialog(context: c, builder:
(dc) => AlertDialog(title: Text(title), content: Text(tr(c, 'are_you_sure')), actions: [
  TextButton(onPressed: () => Navigator.pop(dc), child: Text(tr(c, 'cancel'))),
  FilledButton(style: FilledButton.styleFrom(backgroundColor: Colors.red), onPressed: () { onOk();
Navigator.pop(dc); }, child: Text(title)),
]));

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  await initializeDateFormatting();
  runApp(ChangeNotifierProvider(create: (_) => AppState(), child: const MyApp())); }

class MyApp extends StatelessWidget {
  const MyApp({super.key});
@override Widget build(BuildContext c) {
    final isDark = c.select<AppState, bool>((s) => s.isDark);
    final isAr = c.select<AppState, bool>((s) => s.isAr);
    return MaterialApp(
      debugShowCheckedModeBanner: false, themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
```

```
      theme: ThemeData(useMaterial3: true, colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal)),

      darkTheme: ThemeData(useMaterial3: true, colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal,
brightness: Brightness.dark)),
      builder: (ctx, child) => Directionality(textDirection: isAr ? ui.TextDirection.rtl :
ui.TextDirection.ltr, child: child!),
      home: StreamBuilder<User?>(stream: authService.authStateChanges, builder: (ctx, snap) =>
snap.connectionState == ConnectionState.waiting
          ? const Scaffold(body: Center(child: CircularProgressIndicator())) : (snap.hasData ? const
MainScreen() : const LoginScreen())),
    ); } }

class LoginScreen extends StatefulWidget { const LoginScreen({super.key}); @override State<LoginScreen>
createState() => _LoginScreenState(); }

class _LoginScreenState extends State<LoginScreen> {
  final emailCtrl = TextEditingController(), passCtrl = TextEditingController();
  bool isSignUp = false, loading = false;
  String? error;

  Future<void> submit() async {
    final email = emailCtrl.text.trim(), pass = passCtrl.text.trim();
    if (email.isEmpty || pass.isEmpty) return;
    setState(() { loading = true; error = null; });
    final err = isSignUp ? await authService.signUp(email, pass) : await authService.signIn(email, pass);

    if (!mounted) return;
    setState(() { loading = false; error = err; }); }

@override void dispose() { emailCtrl.dispose(); passCtrl.dispose(); super.dispose(); }

@override Widget build(BuildContext c) => Scaffold(body: Center(child: SingleChildScrollView(padding: const
EdgeInsets.all(24), child: Column(mainAxisSize: MainAxisSize.min, children: [
    Text(isSignUp ? tr(c, 'signup') : tr(c, 'login'), style: const TextStyle(fontSize: 24, fontWeight:
FontWeight.bold)),
    const SizedBox(height: 24),
    TextField(controller: emailCtrl, keyboardType: TextInputType.emailAddress, decoration:
InputDecoration(labelText: tr(c, 'email'), border: const OutlineInputBorder())),
    const SizedBox(height: 12),
    TextField(controller: passCtrl, obscureText: true, decoration: InputDecoration(labelText: tr(c,
'password'), border: const OutlineInputBorder())),
    if (error != null) Padding(padding: const EdgeInsets.only(top: 12), child: Text(error!, style: const
TextStyle(color: Colors.red), textAlign: TextAlign.center)),
    const SizedBox(height: 16),
    FilledButton(onPressed: loading ? null : submit, child: loading ? const SizedBox(width: 18, height: 18,
child: CircularProgressIndicator(strokeWidth: 2)) : Text(isSignUp ? tr(c, 'signup') : tr(c, 'login'))),

    TextButton(onPressed: () => setState(() => isSignUp = !isSignUp), child: Text(isSignUp ? tr(c,
'have_account') : tr(c, 'no_account'))),
  ]))); }

class MainScreen extends StatefulWidget { const MainScreen({super.key}); @override State<MainScreen>
createState() => _MainScreenState(); }

class _MainScreenState extends State<MainScreen> {
  int idx = 0;
  final ctrl = TextEditingController(text: 'a beautiful landscape, highly detailed');
  final api = Api();

@override void dispose() { ctrl.dispose(); super.dispose(); }

  Future<void> generate(BuildContext c) async {
    final s = c.read<AppState>();
    final prompt = ctrl.text.trim();
    if (hasArabic(prompt)) { ScaffoldMessenger.of(c).showSnackBar(SnackBar(content: Text(tr(c,
'arabic_warning')))); return; }
    if (prompt.isEmpty || s.cooldownLeft > 0 || s.loading) return;
    s.set(() => s.loading = true);
    try {
      final bytes = await api.gen(prompt, s.model, s.style);
      final path = await saveImageFile(bytes);
      if (!c.mounted) return;
      await s.addToHistory(GenImg(path, prompt, DateTime.now()));
      s.set(() => s.loading = false); s.startCooldown();
    } catch (e) {
```

```
      s.set(() => s.loading = false);
      if (c.mounted) ScaffoldMessenger.of(c).showSnackBar(SnackBar(content: Text('${tr(c, 'error')}:
${cut(e.toString(), 200)}'), duration: const Duration(seconds: 6))); } }

  Widget _thumb(Future<Uint8List> f, {double? h}) => FutureBuilder<Uint8List>(future: f, builder: (_, snap) =>
snap.hasData
      ? Image.memory(snap.data!, width: h == null ? double.infinity : null, height: h, fit: BoxFit.cover,
gaplessPlayback: true)
      : SizedBox(height: h ?? 200, child: const Center(child: CircularProgressIndicator())));

@override Widget build(BuildContext c) {
    final s = c.watch<AppState>();
    final ar = hasArabic(ctrl.text), onCooldown = s.cooldownLeft > 0;
    if (!s.ready) return const Scaffold(body: Center(child: CircularProgressIndicator()));
    return Scaffold(
      appBar: AppBar(title: Text(tr(c, 'app_title')), actions: [
        IconButton(icon: Icon(s.isDark ? Icons.light_mode : Icons.dark_mode), onPressed: () => s.set(() =>
s.isDark = !s.isDark)),
        IconButton(icon: const Icon(Icons.language), onPressed: () => s.set(() => s.isAr = !s.isAr)),
        if (idx == 1 && s.history.isNotEmpty) IconButton(icon: const Icon(Icons.delete_sweep), onPressed: () =>
confirmDialog(c, tr(c, 'clear_history'), () => s.clearHistory())),
        IconButton(icon: const Icon(Icons.logout), onPressed: () => authService.signOut()),
      ]),
      body: IndexedStack(index: idx, children: [_buildGenTab(c, s, ar, onCooldown), _buildHistoryTab(c, s)]),

      bottomNavigationBar: NavigationBar(selectedIndex: idx, onDestinationSelected: (i) => setState(() => idx =
i), destinations: [
        NavigationDestination(icon: const Icon(Icons.image), label: tr(c, 'generate')),
        NavigationDestination(icon: const Icon(Icons.history), label: tr(c, 'history')),
      ]),
    ); }

  Widget _buildGenTab(BuildContext c, AppState s, bool ar, bool onCooldown) => SingleChildScrollView(padding:
const EdgeInsets.all(16), child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [

    Text(tr(c, 'cooldown_info'), style: const TextStyle(fontSize: 12, color: Colors.grey)),
    const SizedBox(height: 8),
    TextField(controller: ctrl, onChanged: (_) => setState(() {}), maxLines: 3, decoration:
InputDecoration(labelText: tr(c, 'prompt'), border: const OutlineInputBorder(),
      errorText: ar ? tr(c, 'arabic_warning') : null, suffixIcon: IconButton(icon: const Icon(Icons.clear),
onPressed: () => setState(() => ctrl.clear())))),
    const SizedBox(height: 12),
    Wrap(spacing: 8, children: models.keys.map((k) => FilterChip(label: Text(tr(c, 'model_$k')), selected:
s.model == k, onSelected: (_) => s.set(() => s.model = k))).toList()),
    const SizedBox(height: 12),
    Wrap(spacing: 8, children: styles.keys.map((k) => FilterChip(label: Text(tr(c, 'style_$k')), selected:
s.style == k, onSelected: (_) => s.set(() => s.style = k))).toList()),
    const SizedBox(height: 12),
    if (onCooldown) Padding(padding: const EdgeInsets.only(bottom: 12), child: Column(children: [
      Text('${tr(c, 'cooldown_wait')} ${s.cooldownLeft} ${tr(c, 'seconds_suffix')}', style: const
TextStyle(fontWeight: FontWeight.bold)),
      LinearProgressIndicator(value: s.cooldownLeft / Cfg.cooldown),
    ])),
    FilledButton.icon(onPressed: (s.loading || onCooldown) ? null : () => generate(c),
      icon: s.loading ? const SizedBox(width: 18, height: 18, child: CircularProgressIndicator(strokeWidth: 2))
: const Icon(Icons.image), label: Text(tr(c, 'generate'))),

    const SizedBox(height: 16),
    if (s.loading) const Padding(padding: EdgeInsets.only(top: 30), child: Center(child:
CircularProgressIndicator()))
    else if (s.history.isNotEmpty) ...[
      const Divider(),
      Stack(alignment: Alignment.topRight, children: [
        _thumb(s.history.first.bytes),
        Padding(padding: const EdgeInsets.all(8), child: CircleAvatar(backgroundColor: Colors.black54,

          child: IconButton(icon: const Icon(Icons.download, color: Colors.white), onPressed: () async =>
downloadImage(c, await s.history.first.bytes, s.history.first.prompt)))),
      ]),
    ] else Padding(padding: const EdgeInsets.only(top: 50), child: Center(child: Text(tr(c, 'no_images')))),

  ]));

  Widget _buildHistoryTab(BuildContext c, AppState s) => Container(padding: const EdgeInsets.all(8), child:
s.history.isEmpty ? Center(child: Text(tr(c, 'no_images'))) : ListView.builder(
    itemCount: s.history.length,
    itemBuilder: (_, i) {
      final img = s.history[i];
      return Card(child: ListTile(
        leading: SizedBox(width: 60, height: 60, child: _thumb(img.bytes, h: 60)),
        title: Text(cut(img.prompt, 40), maxLines: 1, overflow: TextOverflow.ellipsis),
        subtitle: Text(DateFormat.yMd().add_jm().format(img.time)),
        trailing: Row(mainAxisSize: MainAxisSize.min, children: [
          IconButton(icon: const Icon(Icons.download), onPressed: () async => downloadImage(c, await img.bytes,
img.prompt)),
          IconButton(icon: const Icon(Icons.delete, color: Colors.red), onPressed: () => confirmDialog(c, tr(c,
'delete'), () => s.removeFromHistory(i))),
        ]),
        onTap: () => showDialog(context: c, builder: (dc) => AlertDialog(content: Column(mainAxisSize:
MainAxisSize.min, children: [
          _thumb(img.bytes), const SizedBox(height: 8), Text(img.prompt),
Text(DateFormat.yMd().add_jm().format(img.time)),
        ]), actions: [
          TextButton.icon(icon: const Icon(Icons.download), label: Text(tr(c, 'download')), onPressed: () async
=> downloadImage(c, await img.bytes, img.prompt)),
          TextButton(onPressed: () => Navigator.pop(dc), child: Text(tr(c, 'close'))),
        ])),
      ));
    },
  )); }
```